SIMATS ENGINEERING
ASSIGNMENT 2

COURSE NAME: DATA STRUCTURE
COURSE CODE: CSA0307
NAME / REG NO: MONISH.L [1972260035]

Q1. Describe the precedence and associativity rules for arithmetic operators. How do they affect infix to postfix conversion?

Answer:

Precedence decides which operator is applied first when an expression contains different operators; an operator with higher precedence binds more tightly to its operands. **Associativity** decides the order of evaluation when operators of the **same precedence** appear together: left-to-right (left associative) or right-to-left (right associative).

Rules for arithmetic operators (highest to lowest)

Level	Operators	Associativity	Example
1	( ) parentheses	Innermost first	A * (B + C) evaluates B + C first
2	^ (exponentiation)	Right to left	A ^ B ^ C = A ^ (B ^ C)
3	* / %	Left to right	A / B * C = (A / B) * C
4	+ -	Left to right	A - B + C = (A - B) + C

Parentheses override both rules. (In C there is no exponent operator; ^ here is the usual notation used in infix-to-postfix problems.)

Effect on infix to postfix conversion

In postfix notation the operators come after their operands and no parentheses or precedence rules are needed when the expression is evaluated. The precedence and associativity rules are therefore used **during conversion** to decide the position of each operator. The algorithm uses a stack for operators:

- **Operand:** append it directly to the output.
- **Operator:** while the stack top is an operator with **higher precedence**, or with **equal precedence and the current operator is left associative**, pop it to the output; then push the current operator.
- **Right-associative operator (^):** an equal-precedence operator on the stack is **not** popped, so the new ^ is pushed above it.
- **'(':** push on the stack. **')':** pop operators to the output until '(' is found, then discard both parentheses.
- **End of input:** pop all remaining operators to the output.

How the rules change the result

Infix	Rule applied	Postfix
A + B * C	* has higher precedence, so it is pushed above + and is output before it	A B C * +
(A + B) * C	parentheses force + first	A B + C *
A - B - C	equal precedence, left associative: the first - is popped before the second is pushed	A B - C -
A ^ B ^ C	equal precedence, right associative: the first ^ stays on the stack	A B C ^ ^
A + B * C - D / E	* and / before + and -; the - pops + (equal, left associative)	A B C * + D E / -

Conclusion: precedence determines which operator is emitted first, associativity resolves ties between equal-precedence operators, and parentheses override both. If these rules are not applied, the postfix form would represent a different expression and evaluate to a wrong value.

Q2. Trace the conversion of A * (B + C) to postfix and illustrate how parentheses are handled.

Answer:

The expression is scanned from left to right with an operator stack. Precedence: * is higher than +; (is pushed always and is never popped by an operator, only by).

Step-by-step trace

Step	Symbol	Stack (bottom → top)	Postfix output	Action
1	A	—	A	Operand: add to output
2	*	*	A	Operator: pop higher/equal precedence, push '*'
3	(	*(	A	'(' : push
4	B	*(	AB	Operand: add to output
5	+	*(+	AB	Operator: pop higher/equal precedence, push '+'
6	C	*(+	ABC	Operand: add to output
7	)	*	ABC+	')' : pop to output until '(' ; discard '('
8	end	—	ABC+*	Pop remaining operators

How the parentheses are handled

- ((step 3) is pushed on top of *. Inside the parentheses it acts as a barrier: when + arrives (step 5) the top is (, so nothing is popped and + is pushed above it. Without the barrier * would have been popped before + and the grouping would be lost.
-) (step 7) pops operators until (is found: + goes to the output, giving ABC+, and the (is removed. Parentheses never appear in the postfix output.
- After this the stack holds only *, which is popped at the end of the input (step 8).

Result: the postfix expression is ABC+*.

Verification

Take A = 2, B = 3, C = 4. Infix: 2 * (3 + 4) = 2 * 7 = 14. Evaluate postfix 2 3 4 + * with a stack: push 2, 3, 4; + pops 4 and 3 and pushes 7; * pops 7 and 2 and pushes 14. Both give **14**.

For comparison, without parentheses `A * B + C` becomes `A B * C +` (the `*` is done first), which is a different expression; the parentheses are what make `B + C` evaluated before the multiplication.

Signature of the student:

